

Rapport – TP 1 MERN

Ma première API Express : le service du blog

Mohamed Aziz Mansour – 5ème DS G3

École Polytechnique de Sousse – Dr. Abdelweheb GUEDES – 2026–2027

1. Constats et prédictions vérifiées

Le « 2 » de /api/articles/2 : texte ou nombre ?

Constat	Pourquoi
Du texte (« 2 »)	Tout fragment d'URL parvient au serveur sous forme de chaîne. Sans <code>Number(...)</code> , la comparaison <code>"2" === 2</code> échoue (deux types distincts) : <code>find</code> ne repère rien et l'API répond 404 « Article 2 introuvable ».

Le compteur total avec filtre par auteur

Adresse	Constat	Pourquoi
/api/articles	3	Sans <code>?author</code> , la collection entière est renvoyée.
/api/articles?author=Aya	2	<code>filter</code> ne conserve que les billets 2 et 3.
/api/articles?author=Omar	0	Omar n'a signé aucun billet : <code>filter</code> produit <code>[]</code> , jamais <code>undefined</code> .

Les codes de statut relevés dans Postman

Méthode	Adresse	Corps	Constat	Pourquoi
GET	/api/articles	–	200 OK	Affichage de la collection : tout s'est bien passé.
POST	/api/articles	{ "title": "Mon premier article", "author": "Aziz Mansour" }	201 Created	Les deux champs fournis : le billet numéro 4 est ajouté.
GET	/api/articles/4	–	200 OK	Le billet créé reste accessible tant que le processus tourne.
POST	/api/articles	{ "title": "Sans auteur" }	400 Bad Request	Le champ <code>author</code> est absent : le contrôle bloque la demande.
GET	/api/articles/99	–	404 Not Found	Aucun billet ne porte l'identifiant 99.

Après relance du processus, `GET /api/articles` indique de nouveau `"total": 3` : tout vivait en mémoire vive, ce qui justifie une vraie base de données (MongoDB, séance 3).

GET`http://localhost:3000/api/articles`**Send****RESPONSE****200 OK** 2 ms

```
{
  "total": 3,
  "articles": [
    {
      "id": 1,
      "title": "Bienvenue sur le blog",
      "author": "Admin"
    },
    {
      "id": 2,
      "title": "Mon premier serveur Express",
      "author": "Aya"
    },
    {
      "id": 3,
      "title": "Tester une API avec Postman",
      "author": "Aya"
    }
  ]
}
```

Mohamed Aziz Mansour – TP1 MERN – API blog Express

GET /api/articles → 200 OK

POST

http://localhost:3000/api/articles

Send

REQUEST BODY · RAW JSON

```
{
  "title": "Mon premier article",
  "author": "Aziz Mansour"
}
```

RESPONSE

201 Created29 ms

```
{
  "message": "Article créé",
  "article": {
    "id": 4,
    "title": "Mon premier article",
    "author": "Aziz Mansour"
  }
}
```

Mohamed Aziz Mansour – TP1 MERN – API blog Express

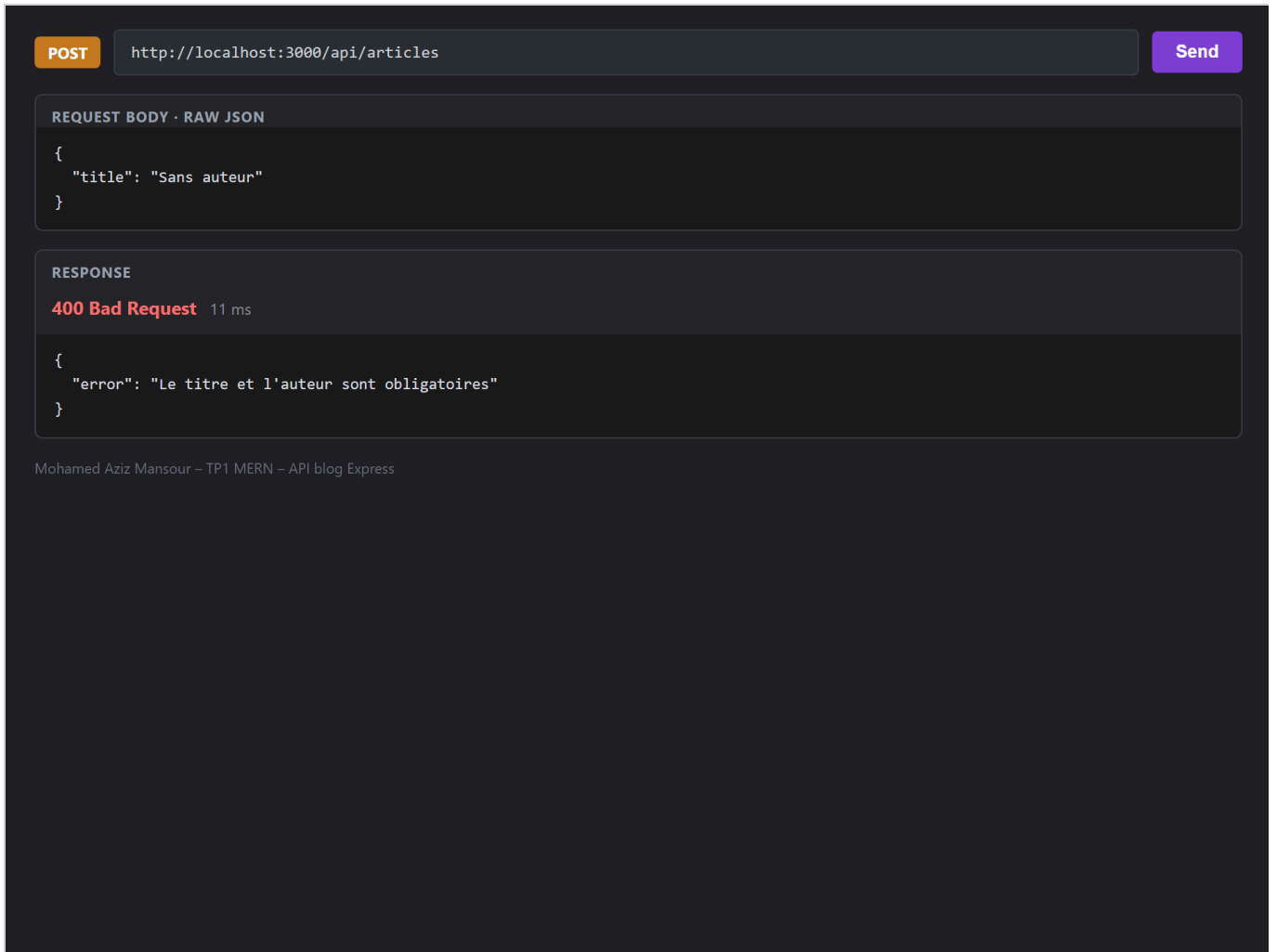
POST /api/articles → 201 Created

GET`http://localhost:3000/api/articles/4`**Send****RESPONSE****200 OK** 4 ms

```
{
  "id": 4,
  "title": "Mon premier article",
  "author": "Aziz Mansour"
}
```

Mohamed Aziz Mansour – TP1 MERN – API blog Express

GET /api/articles/4 → 200 OK



POST /api/articles → 400 Bad Request

2. Bases JavaScript (second exercice)

Fichier notions-js.js, exécuté avec `node notions-js.js`.

```
const materiel = [
  { etiquette: "Casque", cout: 65 },
  { etiquette: "Webcam", cout: 210 },
  { etiquette: "Tapis", cout: 18 },
];

// 1. Déstructuration : récupérer deux propriétés du premier élément
const { etiquette, cout } = materiel[0];
console.log(etiquette, cout);

// 2. find : retrouver l'élément appelé 'Tapis' puis afficher son tarif
const tapis = materiel.find((m) => m.etiquette === "Tapis");
console.log(tapis.cout);

// 3. filter : garder uniquement le matériel à moins de 100
const abordables = materiel.filter((m) => m.cout < 100);
console.log(abordables);

// 4. Fonction fléchée : applique une remise de 10 %
const appliquerRemise = (montant) => montant * 0.9;
console.log(appliquerRemise(210));
```

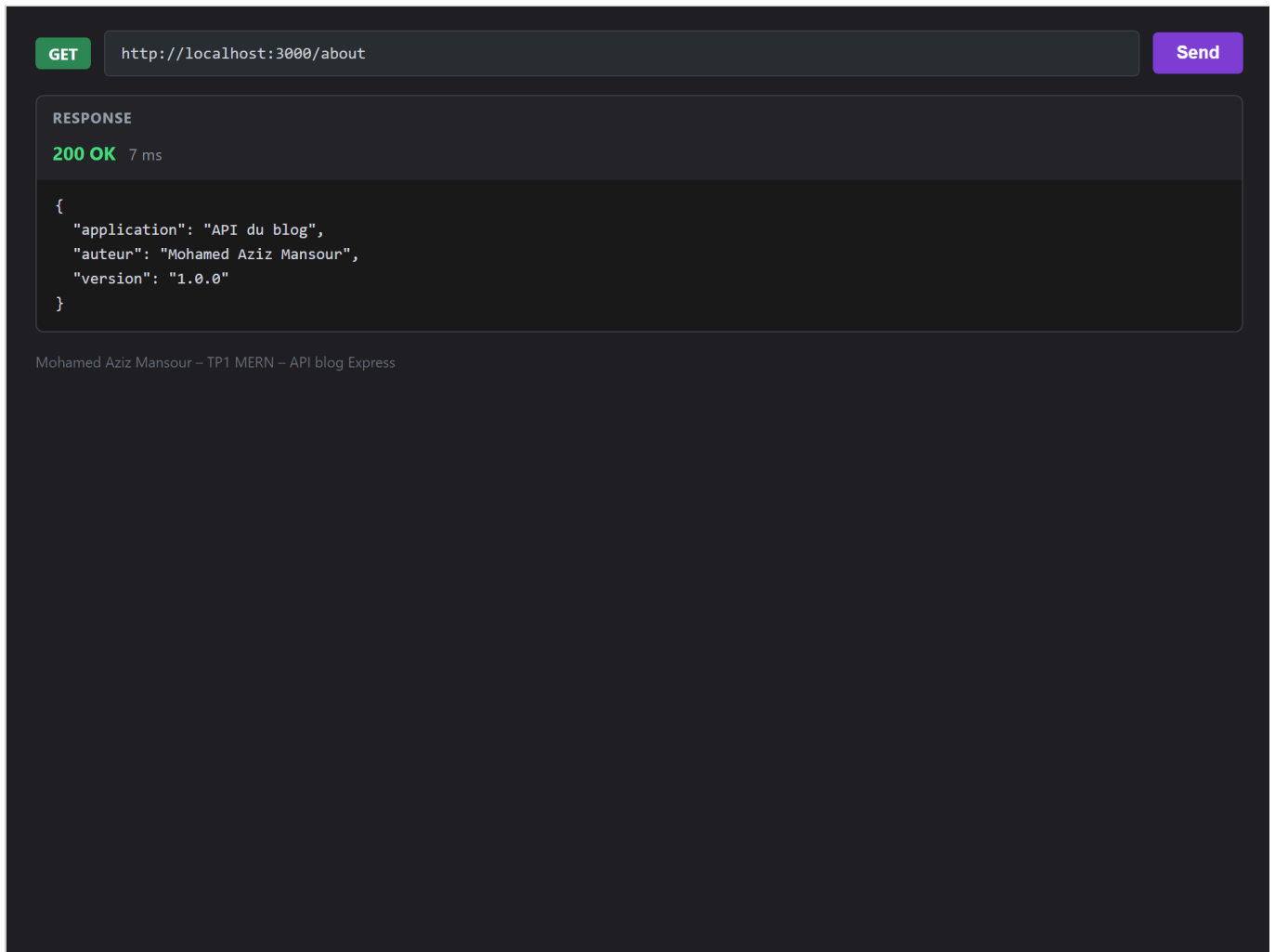
Affichage recueilli dans le terminal :

```
Casque 65
18
[ { etiquette: 'Casque', cout: 65 }, { etiquette: 'Tapis', cout: 18 } ]
189
```

3. Extensions de l'API (premier exercice)

1. GET /about

```
// introuvable : serveur.get('/about')
```



The screenshot shows a REST client interface with a dark theme. At the top, there is a green 'GET' button and a text input field containing 'http://localhost:3000/about'. To the right of the input field is a purple 'Send' button. Below the input field, the word 'RESPONSE' is displayed in white. Underneath, the status '200 OK' is shown in green, followed by '7 ms' in white. The response body is a JSON object displayed in a light gray box:

```
{  "application": "API du blog",  "auteur": "Mohamed Aziz Mansour",  "version": "1.0.0"}
```

 At the bottom left of the interface, the text 'Mohamed Aziz Mansour – TP1 MERN – API blog Express' is visible.

GET /about → 200 : le nom du service, son auteur et sa version.

2. GET /api/users

```
const membres = [  
  { id: 1, name: "Aya", email: "aya@example.com" },  
  { id: 2, name: "Omar", email: "omar@example.com" },  
  { id: 3, name: "Youssef", email: "youssef@example.com" },  
];  
  
// introuvable : serveur.get('/api/users')
```

The screenshot shows a REST client interface with a dark theme. At the top, there is a green 'GET' button and a text input containing the URL 'http://localhost:3000/api/users'. To the right of the input is a purple 'Send' button. Below the input, the word 'RESPONSE' is displayed in white. Underneath, the status '200 OK' is shown in green, followed by '5 ms' in white. The main area contains a JSON response in white text on a dark background. At the bottom left, there is a small text attribution: 'Mohamed Aziz Mansour – TP1 MERN – API blog Express'.

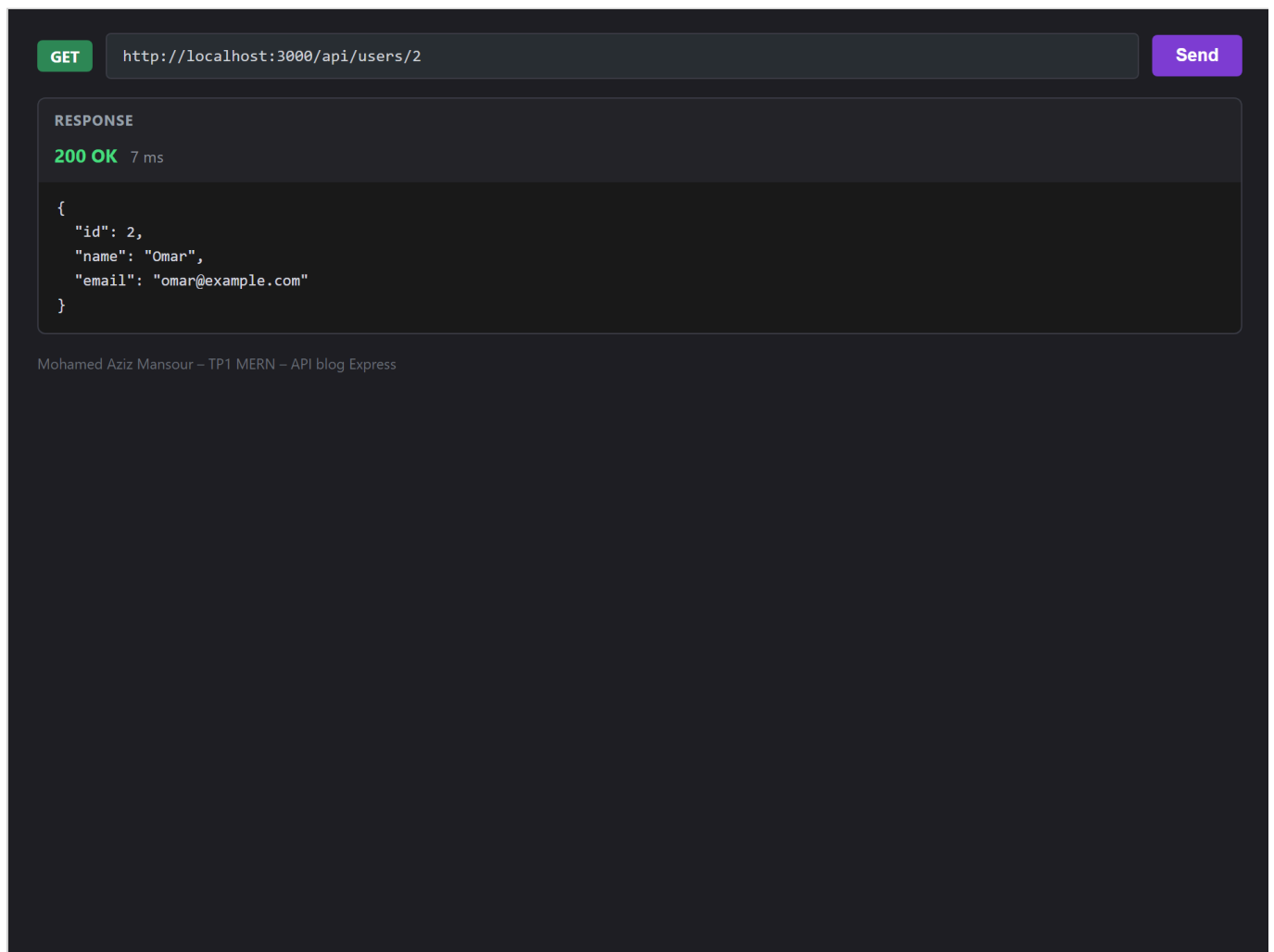
```
{  
  "total": 3,  
  "users": [  
    {  
      "id": 1,  
      "name": "Aya",  
      "email": "aya@example.com"  
    },  
    {  
      "id": 2,  
      "name": "Omar",  
      "email": "omar@example.com"  
    },  
    {  
      "id": 3,  
      "name": "Youssef",  
      "email": "youssef@example.com"  
    }  
  ]  
}
```

Mohamed Aziz Mansour – TP1 MERN – API blog Express

GET /api/users → 200 : les trois membres stockés dans la constante membres.

3. GET /api/users/:id

```
// introuvable : serveur.get('/api/users/:id')
```

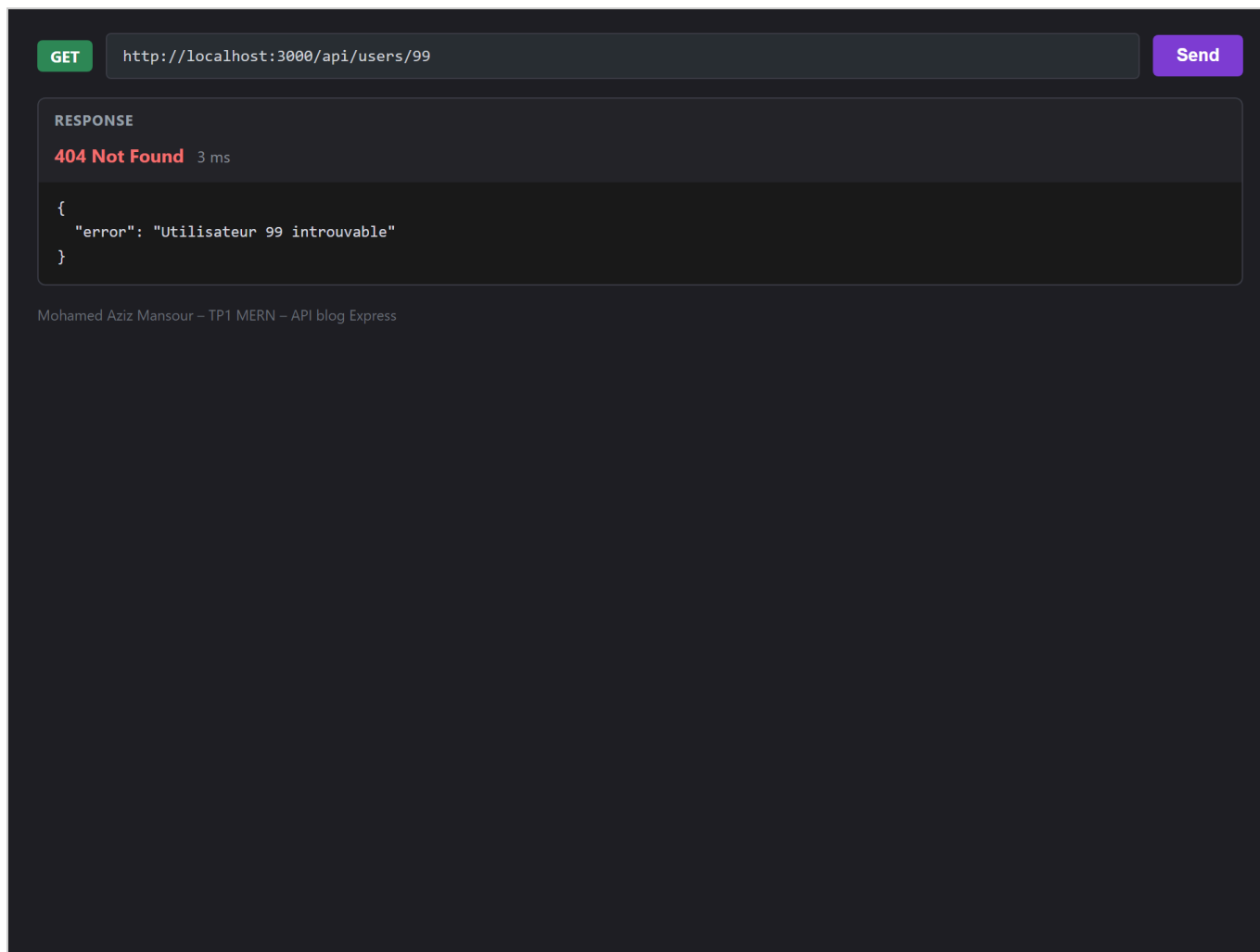


The screenshot shows a REST client interface with a dark theme. At the top, there is a green 'GET' button, a text input field containing the URL 'http://localhost:3000/api/users/2', and a purple 'Send' button. Below this, a 'RESPONSE' section is expanded, showing a status of '200 OK' in green and a response time of '7 ms'. The response body is a JSON object:

```
{  "id": 2,  "name": "Omar",  "email": "omar@example.com"}
```

. At the bottom left of the interface, the text 'Mohamed Aziz Mansour – TP1 MERN – API blog Express' is visible.

Cas favorable : GET /api/users/2 → 200. Number() transforme « 2 » en 2 et find retrouve Omar.



Cas défavorable : GET /api/users/99 → 404. find rend undefined, le return expédie l'erreur et interrompt la fonction.

4. POST /contact

```
// introuvable : serveur.post('/contact')
```

The screenshot shows a REST client interface with a dark theme. At the top, there is a 'POST' method selector and a URL input field containing 'http://localhost:3000/contact'. To the right of the URL is a 'Send' button. Below the URL bar, there are two expandable sections. The first section, titled 'REQUEST BODY - RAW JSON', is expanded and shows a JSON object:

```
{  "email": "aya@example.com",  "message": "Bonjour !"}
```

. The second section, titled 'RESPONSE', is also expanded and shows a '200 OK' status with a response time of '191 ms'. Below the status, it displays a JSON object:

```
{  "message": "Merci, votre message a bien été reçu"}
```

. At the bottom left of the interface, there is a footer text: 'Mohamed Aziz Mansour – TP1 MERN – API blog Express'.

Cas favorable : courriel et contenu fournis → 200 accompagné du remerciement.

POST

http://localhost:3000/contact

Send

REQUEST BODY · RAW JSON

```
{  
  "email": "aya@example.com"  
}
```

RESPONSE

400 Bad Request

11 ms

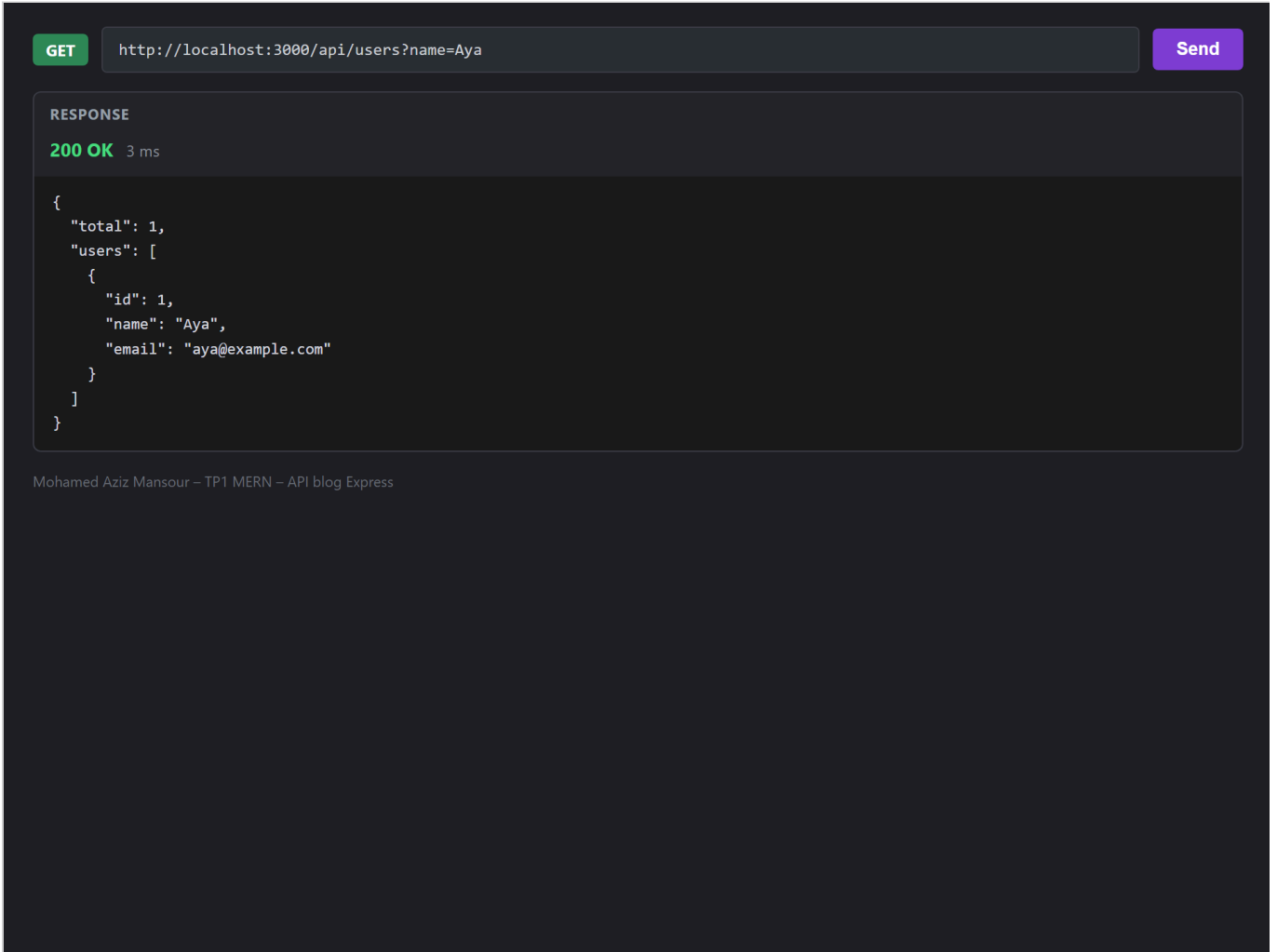
```
{  
  "error": "L'email et le message sont obligatoires"  
}
```

Mohamed Aziz Mansour – TP1 MERN – API blog Express

Cas défavorable : le champ message est absent → 400.

5. Bonus : GET /api/users?name=Aya

```
// même gestionnaire que la question 2 : le tri passe par req.query.name  
// introuvable : serveur.get('/api/users')
```



GET http://localhost:3000/api/users?name=Aya

Send

RESPONSE

200 OK 3 ms

```
{  
  "total": 1,  
  "users": [  
    {  
      "id": 1,  
      "name": "Aya",  
      "email": "aya@example.com"  
    }  
  ]  
}
```

Mohamed Aziz Mansour – TP1 MERN – API blog Express

GET /api/users?name=Aya → 200 avec total: 1 : filter ne conserve qu'Aya.

4. Sources complètes

Dépôt Git : <https://github.com/Kirazul/MERN-1>